

Docker Fundamentals

Docker, Installation, Images, and Containers

Concepts adapted from *Docker Deep Dive*

Learning Objectives

By the end of this lesson, you should be able to:

- Explain what Docker is and why developers use it.
- Describe the main parts of the Docker platform.
- Install and verify Docker on Windows, macOS, or Linux.
- Explain the difference between an image and a container.
- Run, inspect, stop, and remove a basic container.

Main idea

Docker packages an application and its dependencies into a portable image, then runs that image as an isolated container.

The Deployment Problem

Common problem

An application works on the developer's computer but fails on another computer or server.

Docker's approach

Package the application with the environment it needs and run that package consistently.

Possible causes include:

- Different operating-system libraries
- Missing software dependencies
- Different configuration files
- Conflicting runtime versions

What Is Docker?

Definition

Docker is a platform for building, distributing, and running applications in containers.

Build

Package application code, runtime components, libraries, and configuration into an image.

Share

Store and distribute images through a registry such as Docker Hub.

Run

Start one or more isolated containers from the same image.

Dockerfile $\longrightarrow$ Image $\longrightarrow$ Container

Why Is Docker Useful?

Consistency

The same image can be used during development, testing, and deployment.

Efficiency

Containers normally require fewer resources than complete virtual machines.

Portability

A containerized application can run on any compatible Docker host.

Repeatability

A versioned image provides a reusable and predictable application package.

Isolation

Applications run in separate environments while sharing the host operating-system kernel.

Scalability

Multiple containers can be created from one image when more application instances are needed.

Docker Architecture

- The client sends commands to the Docker daemon.
- The daemon creates and manages images, containers, networks, and volumes.
- A registry provides images that can be pulled to a host.

First Test: Hello World

```
docker run --rm hello-world
```

When this command is executed, Docker:

1. Checks whether the `hello-world` image exists locally.
2. Pulls the image from a registry if it is not available.
3. Creates a new container from the image.
4. Starts the container's default process.
5. Displays the output in the terminal.
6. Removes the container after it exits because of `-rm`.

Key observation

The image is the packaged template. The container is the runtime instance created from that template.

Understanding Docker Images

Docker image

A Docker image is an immutable package containing the application, runtime components, libraries, configuration, and metadata required to create a container.

- Images are built from one or more read-only layers.
- Images can be downloaded from or uploaded to registries.
- Repositories group related images.
- Tags identify image variants or versions.
- One image can create many independent containers.

Image name

In `nginx:alpine`, `nginx` is the repository name and `alpine` is the tag.

Images and Layers

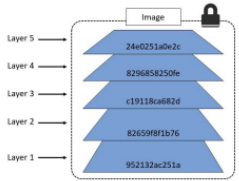


Image layers are read-only and can be shared between images and containers.

```
docker pull nginx:alpine
docker images
docker image inspect nginx:alpine
docker image history nginx:alpine
```

Why layers matter

Layer reuse reduces duplicated storage and can make image distribution more efficient.

Understanding Docker Containers

Docker container

A container is a runnable instance of an image with an isolated process, filesystem view, network configuration, and a thin writable layer.

A container can be:

- Created from an image
- Started, stopped, and restarted
- Inspected while running or stopped
- Connected to networks and persistent volumes
- Removed when it is no longer needed

Container lifetime

A container normally remains active while its main process is running. When that process exits, the container stops.

Container Example: Ubuntu

Start an interactive Ubuntu container:

```
docker run -it --name ubuntu-lab ubuntu:24.04 bash
```

Inside the container:

```
cat /etc/os-release
echo "Hello from the container"
exit
```

Inspect and restart the stopped container:

```
docker ps -a
docker start -ai ubuntu-lab
```

Remove the container when finished:

```
docker rm ubuntu-lab
```

Image vs. Container

Feature	Image	Container
Purpose	Template used to create containers	Runnable instance of an image
State	Immutable and read-only	Has a writable container layer
Execution	Does not execute by itself	Runs one or more processes
Storage	Stored in the local image cache or registry	Stored on a Docker host
Multiplicity	One image can be reused many times	Each container has its own identity and state
Lifecycle	Pull, build, tag, push, remove	Create, start, stop, restart, remove
Analogy	Class or installation package	Object or installed running application

End-to-End Example: Nginx

Run a web server in the background:

```
docker run -d --name web -p 8080:80 nginx:alpine
```

Docker creates a container from the `nginx:alpine` image and maps host port 8080 to container port 80.

```
docker ps
docker logs web
curl http://localhost:8080
```

Stop and remove the container:

```
docker stop web
docker rm web
```

The image remains available after the container is removed:

```
docker images nginx
```

Example for chatbot app

build the image

```
docker build -t chatbot-api:1.0 .
```

check if image exists

```
docker image ls chatbot-api
```

run the container

```
docker run --rm \
  --name chatbot-api \
  -p 8000:8000 \
  --env-file .env \
  chatbot-api:1.0
```

Useful pattern

Use `docker run -rm IMAGE` for short-lived containers that should be removed automatically after they stop.

Push to dockerhub

login

```
docker login --username yourusername
```

push the image

```
docker push yourusername/chatbot-api
    :1.0
```

tag the image

```
docker tag chatbot-api:1.0 yourusername
    /chatbot-api:1.0
```

pull image on another computer

```
docker pull yourusername/chatbot-api
    :1.0
```

Notes

If the Dockerhub repo is private, login before pulling.

The docker image should NOT contain .env, you must create .env on the other computer